



python

Apprendre
à coder
avec



Pr. Houda BENHAR
Année universitaire 2023-2024

PLAN

- Introduction
- Python IDEs
- Installer python
- Variables
- Fonction print
- Fonction input
- Conversion de types
- Instructions
- Expressions numériques



INTRODUCTION

- Python est un langage de programmation populaire créé par Guido van Rossum et sorti en 1991
- Le nom Python vient de la comédie britannique **Monty Python's Flying Circus** et n'a rien à voir avec le serpent du même nom
- L'idée c'était de capturer un air de jeu
- Un langage de programmation qui ne serait pas si difficile, amusant, agréable à utiliser mais en même temps puissant
- Python a une syntaxe simple similaire à la langue anglaise
- Ecrire des programmes avec moins de lignes de code que certains autres langages de programmation



INTRODUCTION

```
#include <stdio.h>
int main(void)
{
    printf("Hello, world!");
}
```

C

```
#include <iostream.h>
int main()
{
    std::cout << "Hello, world! ";
    return 0;
}
```

C++

```
class HelloWorld {
    public static void main(String[]
args) {
        System.out.println("Hello,
World!");
    }
}
```

Java

```
print "Hello, world!"
```

Python



INTRODUCTION

Les domaines d'application les plus connus et avec la plus grande demande en professionnels sont:

Le Machine Learning et l'intelligence artificielle

Data Science et Big Data

Développement Web

Développement d'applications mobiles

Développement de jeux



INTRODUCTION

Python est un langage multiplateforme: les programmes écrits en Python peuvent être exécutés sur différentes plateformes informatiques sans nécessiter de modifications importantes



INTRODUCTION

Python est un langage de programmation multiparadigme: langage polyvalent qui prend en charge plusieurs paradigmes de programmation



Impérative

Programme écrit sous forme d'une séquence d'instructions qui seront exécutées les unes après les autres



Fonctionnelle

L'utilisation de fonctions (ou procédures) pour organiser le code de manière structurée

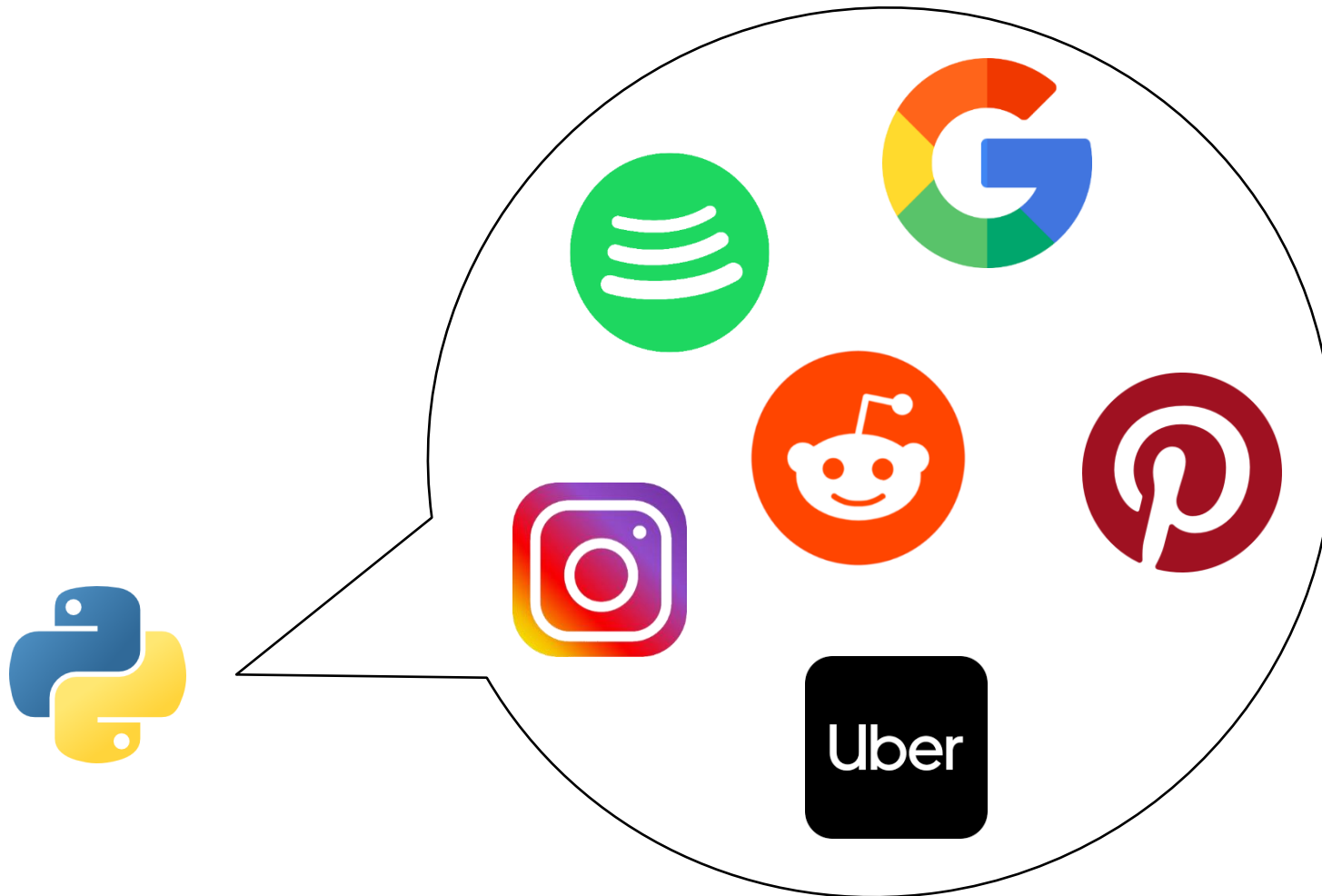


Orientée objet

Les données et les fonctionnalités sont regroupées en objets



INTRODUCTION- QUI UTILISE PYTHON?



IDE'S POUR PYTHON



PyCharm



IDLE



Jupyter notebook

- Basé sur le web
- Plus qu'un IDE
- Code, texte et images
- Outil de présentation ou d'éducation



Spyder



Atom



INSTALLER PYTHON

1. Télécharger Python 3.x sur <https://www.python.org/downloads/>
2. Installer Python sur votre machine. assurez-vous de cocher « " afin de pouvoir ajouter Python à la variable PATH. Add Python 3.x to PATH
3. Pour exécuter votre programme tapez 'python' à l'invite de commande ou ouvrez IDLE pour ouvrir l'interpreteur Python puis tapez votre code. Appuyez sur Entrée.

OU

Ecrivez votre programme dans un fichier et enregistrez avec l'extention **.py**.

Exécutez ce script en tapant **python** nomFichier.py



INSTALLER ANACONDA

1. Télécharger Anaconda sur: <https://www.anaconda.com/products/distribution>
2. Lancez l'installation en double-cliquant sur le fichier d'installation qui s'est téléchargé.
3. Répondez aux questions qui vous sont posées. Les options par défaut sont en général acceptables, pas besoin de les modifier.
4. Une fois l'installation terminée, vérifiez que tout s'est bien passé en lançant le programme Jupyter.

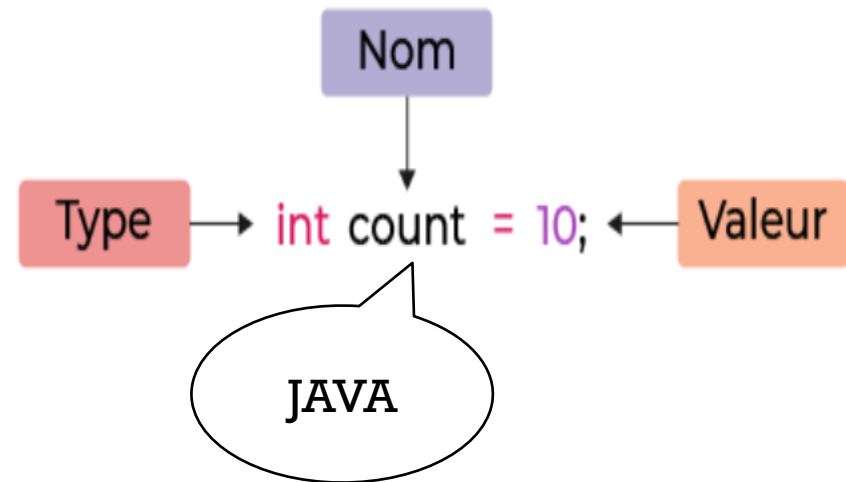


VARIABLES

- La capacité de manipuler des variables est l'une des fonctionnalités les plus puissantes d'un langage de programmation
- Une variable est un nom qui fait référence à une valeur stockée en mémoire

```
>>> message = 'Welcome to becoming a pythonista!'
>>> n = 17
>>> pi = 3.1415926535897931
```

Python



VARIABLES

- Pour afficher la valeur d'une variable, vous pouvez utiliser la fonction print:

```
>>> print(message)
Welcome to becoming a pythonista!
>>> print(pi)
3.141592653589793
```

- Le type d'une variable est le type de la valeur à laquelle elle se réfère.

```
>>> type(message)
<class 'str'>
>>> type(n)
<class 'int'>
>>> type(pi)
<class 'float'>
```



NOMS DE VARIABLES ET MOTS-CLÉS

Le nom d'une variable peut-être ce que vous voulez mais il y a des règles à respecter:

- Un nom de variable en python **peut être défini en lettres minuscules, en majuscules, avec les entiers de 0 à 9.**
- Le caractère (_) peut apparaître dans un nom.
- Un nom de variable peut commencer par une lettre, par un (_) mais pas par un chiffre.
- Les **noms de variable** ne doivent pas contenir d'espaces.
- Un nom de variable prend en compte la casse : ID ≠ id.

Choisir des noms significatifs qui décrivent à quoi sert la variable → Des noms mnémoniques



NOMS DE VARIABLES ET MOTS-CLÉS

- Si vous donnez un nom illégal à une variable, vous obtenez une erreur de syntaxe :

```
>>> 23violins = "Skyfall"  
SyntaxError: invalid syntax  
>>> number@ = 23  
SyntaxError: invalid syntax  
>>> class = "Shakespearean drama"  
SyntaxError: invalid syntax
```

23violins est illégal car il commence par un chiffre. number@ est illégal car il contient un caractère illégal, @.

class est l'un des mots clés de Python.



NOMS DE VARIABLES ET MOTS-CLÉS

- L'interprète utilise des mots clés pour reconnaître la structure du programme, et ils ne peuvent pas être utilisés comme variable des noms.
- Python réserve 35 mots-clés :

<code>and</code>	<code>del</code>	<code>from</code>	<code>None</code>	<code>True</code>
<code>as</code>	<code>elif</code>	<code>global</code>	<code>nonlocal</code>	<code>try</code>
<code>assert</code>	<code>else</code>	<code>if</code>	<code>not</code>	<code>while</code>
<code>break</code>	<code>except</code>	<code>import</code>	<code>or</code>	<code>with</code>
<code>class</code>	<code>False</code>	<code>in</code>	<code>pass</code>	<code>yield</code>
<code>continue</code>	<code>finally</code>	<code>is</code>	<code>raise</code>	<code>async</code>
<code>def</code>	<code>for</code>	<code>lambda</code>	<code>return</code>	<code>await</code>

Si l'interprète se plaint de l'un de vos noms de variables et vous ne savez pas pourquoi, voyez s'il est sur cette liste!



FONCTION PRINT

- L'utilisation des simples quotes:

```
>>> print('l'homme)
SyntaxError: invalid syntax
```

```
>>> print('l\'homme')
l'homme
```

- L'utilisation des doubles quotes:

```
>>> print("l'homme")
l'homme
```

- Afficher le contenu de plusieurs variables:

```
>>> age = 24
>>> nom = "Fatima"
>>> print(nom, "a", age, "ans")
Fatima a 24 ans
```



FONCTION PRINT

- L'utilisation des f-strings ou l'écriture formatée: chaîne de caractères précédée du caractère *f* sans espace entre les deux:

```
>>> age = 24
>>> nom = "Fatima"
>>> print(f"{nom} a {age} ans")
Fatima a 24 ans
```

- Première remarque, la syntaxe apparaît plus lisible que l'équivalent vu ci-avant
- Il ne faut pas omettre le *f* avant le premier guillemet, sinon Python prendra cela pour une chaîne de caractères normale et ne mettra pas en place ce mécanisme de remplacement:

```
>>> print("{nom} a {age} ans")
{nom} a {age} ans
```



FONCTION INPUT

- **input()** permet l'interaction avec l'utilisateur
- Peut être appelée en laissant les parenthèses vides, ou on peut aussi y placer en argument un message explicatif destiné à l'utilisateur:

```
>>> nombre_positif = input("Veuillez entrer un nombre positif quelconque: ")
Veuillez entrer un nombre positif quelconque: 23
>>> print(nombre_positif)
23
```

- **Exercice :** En utilisant les fonctions **input** et **print** proposer un programme en python qui permet à l'utilisateur d'entrer son nom et son passe-temps favori. Le programme affichera ensuite un message:

x aime y pour occuper son temps libre



FONCTION INPUT

- **Correction :**

```
>>> nom = input("Veuillez saisir votre nom! ")
Veuillez saisir votre nom! Houda
>>> passe_temps = input("Quel est votre passe-temps favori? ")
Quel est votre passe-temps favori? cuisiner
>>> print(f"{nom} aime {passe_temps} pour occuper son temps libre")
Houda aime cuisiner pour occuper son temps libre
```



FONCTION INPUT

- Faire des calculs avec la valeur saisie:

```
>>> annee_naissance = input("Quelle est votre année de naissance? ")
Quelle est votre année de naissance? 2013
>>> age = 2022 - annee_naissance
Traceback (most recent call last):
  File "<pyshell#36>", line 1, in <module>
    age = 2022 - annee_naissance
TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

- Conversion de type

```
>>> annee_naissance = input("Quelle est votre année de naissance? ")
Quelle est votre année de naissance? 2013
>>> age = 2022 - int(annee_naissance)
>>> print(age)
9
```



CONVERSIONS DE TYPES

- Python définit des fonctions de conversion de type pour convertir directement un type de données en un autre.

```
>>> user_value = input("Veuillez saisir un nombre : ")
Veuillez saisir un nombre : 3.14159
>>> type(user_value)
<class 'str'>
>>> user_value = float(user_value)
>>> type(user_value)
<class 'float'>
>>> print(user_value)
3.14159
```

- On utilise le nom du type de destination (`int()`, `float()`, `str()..`) pour produire une nouvelle donnée du type souhaité

! Si une chaîne de caractères contient la représentation d'une valeur numérique, il est alors possible de convertir cette chaîne en une donnée numérique.



INSTRUCTIONS

- Nous avons vu deux types d'instructions:
 - Instruction d'affectation
 - Instruction print/input
- Une instruction est une unité de code que l'interpréteur Python peut exécuter.
- Une instruction Python se termine par un retour à la ligne.
- Cela signifie que chaque ligne d'un script Python est une instruction.
- Mais nous pouvons faire en sorte qu'une instruction s'étende sur plusieurs lignes avec le caractère de continuation de ligne (\).

```
>>> a = 1 + 2 + 3 + \  
      4 + 5 + 6 + \  
      7 + 8 + 9
```



INSTRUCTION D'AFFECTATION

- Nous attribuons une valeur à une variable à l'aide de l'opérateur d'affectation (=)
- Une instruction d'affectation se compose d'une **expression** sur le côté droit et d'une **variable** pour stocker le résultat

```
message = "Python is fun"  
x = x + 1  
x = 3.9 * x * (1 - x)
```

Une expression est une combinaison de valeurs, de variables, d'opérateurs et d'appels de fonction qui produit ou renvoie toujours une valeur de résultat.



INSTRUCTION D'AFFECTATION

- Sous Python, on peut assigner une valeur à plusieurs variables simultanément:

```
>>> x = y = 9
>>> x
9
>>> y
9
```

- On peut aussi effectuer des affectations parallèles à l'aide d'un seul opérateur:

```
>>> a, b, c = 4, 8.33, "test"
>>> a
4
>>> b
8.33
>>> c
'test'
```



EXPRESSIONS NUMERIQUES

```
>>> xx = 2
>>> xx = xx + 2
>>> print(xx)
4
>>> yy = 440 * 12
>>> print(yy)
5280
>>> zz = yy / 1000
>>> print(zz)
5.28
```

```
>>> jj = 23
>>> kk = jj % 5
>>> print(kk)
3
>>> print(4 ** 3)
64
>>> print(23 // 5)
4
```

Les opérateurs arithmétiques

Opérateur	Opération
+	Addition
-	Soustraction
*	Multiplication
/	Division
**	Puissance
%	Modulo
//	Division entière



ORDRE DES OPÉRATIONS

- Sous Python, les règles de priorité sont les mêmes que celles qui vous ont été enseignées au cours de mathématique.

PEMDAS:

- **P** pour parentheses: Ce sont elles qui ont la plus haute priorité. Elles vous permettent donc de « forcer » l'évaluation d'une expression dans l'ordre que vous voulez.
- **E** pour exposants.
- **M** et **D** pour multiplication et division, qui ont la même priorité.
- **A** et **S** pour addition et soustraction, qui ont la même priorité.

Si deux opérateurs ont la même priorité, l'évaluation est effectuée de gauche à droite.



EXPRESSIONS- EXERCICE

Parentheses

Exposant

Multiplication, division

Addition, soustraction

$X = 1 + 2 ** 3 / 4 * 5$

```
>>> x = 1 + 2 ** 3 / 4 * 5
```

```
>>> print(x)
```

```
11.0
```

$1 + 2 ** 3 / 4 * 5$

$1 + 8 / 4 * 5$

$1 + 2 * 5$

$1 + 10$

11



EXPRESSIONS- EXERCICE

Supposons que nous exécutons les instructions d'affectation suivantes:

```
largeur = 17  
hauteur = 12.0
```

Pour chacune des expressions suivantes, écrivez la valeur de l'expression et le type (de la valeur de l'expression).

1. `largeur // 2` → 8 <class 'int'>
2. `2.largueur / 2.0` → 8.5 <class 'float'>
3. `3.hauteur / 3` → 4.0 <class 'float'>
4. `4.largueur % 2` → 1 <class 'int'>

Utilisez l'interpréteur Python pour vérifier vos réponses.



EXPRESSIONS- EXERCICE

Prédire le résultat: opérations et conversions de types.

Essayez de prédire le résultat de chacune des instructions suivantes, puis vérifiez-le dans l'interpréteur Python:

- $(1+2)**3$ → 27
- `"Da" * 4` → 'DaDaDaDa'
- `"Da" + 3` → `TypeError`
- $(\text{"Pa"+"La"}) * 2$ → 'PaLaPaLa'
- $(\text{"Da"}*4) / 2$ → `TypeError`
- $5 / 2$ → 2.5
- $5 // 2$ → 2
- $5 \% 2$ → 1
- `str(4) * int("3")` → '444'
- `int("3") + float("3.2")` → 6.2
- `str(3) * float("3.2")` → `TypeError`
- `str(3/4) * 2` → '0.750.75'



EXPRESSIONS- EXERCICE

- Ecrivez un programme qui demande à l'utilisateur de saisir une temperature Fahrenheit et la convertit en température Celsius :

$$^{\circ}\text{C} = (^{\circ}\text{F} - 32.0) * 5.0 / 9.0$$

```
>>> inp = input('Enter Fahrenheit Temperature: ')
Enter Fahrenheit Temperature: 72
>>> fahr = float(inp)
>>> cel = (fahr - 32.0) * 5.0 / 9.0
>>> print(cel)
22.22222222222222
```



COMMENTAIRES

- Un commentaire c'est du texte commençant par le symbole **#** qui va être ignoré par l'interpreteur.
- Les commentaires peuvent servir à expliquer une partie compliquée d'un programme, désactiver temporairement une ligne de code, ou à mettre des indications dans le code, comme son auteur ou sa date.
- Un commentaire peut apparaître au début d'une ligne ou à la suite d'un espace ou de code, mais pas à l'intérieur d'une chaîne de caractères littérale:

```
# this is the first comment  
spam = 1 # and this is the second comment  
        # ... and now a third!  
text = "# This is not a comment because it's inside quotes."
```



COMMENTAIRES

- Les commentaires sont plus utiles lorsqu'ils documentent des fonctionnalités non évidentes du code.

- Ce commentaire est redondant avec le code et inutile:

```
v = 5 # assigner 5 à v
```

- Ce commentaire contient des informations utiles qui ne sont pas dans le code:

```
v = 5 # vitesse en mètres/seconde
```

! N'utilisez donc de commentaires que lorsque cela s'avère nécessaire

! La priorité est toujours d'écrire le code le plus clair possible: il est très souvent bien plus efficace de comprendre un programme directement à partir de ses instructions, qu'en alternant la lecture des instructions avec celle d'explications spécifiques.



COMMENTAIRES

- Une variante consiste à commencer un commentaire avec trois guillemets `"""` et à le finir de la même façon. C'est une façon aisée d'écrire un long commentaire qui court sur plusieurs lignes

```
"""  
This is a comment  
written in  
more than just one line  
"""
```

